

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание классов

Студент гр. 4381

Валиев Р.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

Цель работы

Цель работы разработать систему классов для моделирования игры

Задание.

На 6/3/1 баллов:

1. Создать класс(ы) игры, который реализует основной цикл игры, и которому передаются команды от пользователя. Игровой цикл состоит из следующих шагов:

- a. Начало игры
- b. Запуск уровня
- c. Ход игрока. Ход, атака или применение заклинания
- d. Ход союзников – если имеются
- e. Ход врагов
- f. Ход вражеской базы и башни – если имеются

Условие прохождения уровня студент определяет самостоятельно. Если игрок проигрывает, то игроку должно предлагаться начать заново игру, либо выйти из программы. Все взаимодействие должно происходить через классы игры.

2. Реализовать систему сохранения и загрузки игры. Пользователь должен иметь возможность сохранить игру в любой момент. Пользователь должен иметь возможность загружаться при запуске программы (или выбрать новую), либо во время игры. Сохранения должны оставаться в консистентном состоянии между запусками игры.

3. Добавить обработку исключительных ситуаций для загрузки/сохранения, например, невозможность записать в файл, нельзя загрузиться так как файл не существует или в нем некорректные данные.

На 8/4/1.5 баллов:

4. Реализовать переход на следующий уровень, после прохождения уровня. При переходе на следующий уровень создается новое поле другого размера с более сильными врагами. При переходе на следующий уровень, значение жизни игрока восстанавливается, и половина его карточек заклинаний случайным образом удаляется.

На 10/5/2 баллов:

5. Реализовать прокачку игрока при переходе между уровнями. Пользователь может улучшить характеристики игрока или улучшить заклинание (что и как улучшать определяет студент). Для этого нужно расширить игровой цикл.

Примечания:

Класс игры может знать о игровых сущностях, но не наоборот. При работе с файлом используйте идиому RAII. Исключения должны обязательно обрабатываться, и программа не должна завершаться.

Исключения должны быть информативными (содержать информацию о том, что и где произошло), на разные виды исключительных ситуаций должны быть свои исключения.

Выполнение работы.

1. Создать класса игры, который реализует основной цикл игры

Класс GameControl создан (core/GameControl.cpp, core/GameControl.h). Класс управляет игровым циклом и содержит следующие компоненты:

- unique_ptr<Player> — игрок
- unique_ptr<Board> — игровое поле
- unique_ptr<Hand> — рука заклинаний
- LevelManager — менеджер уровней

Игровой цикл реализован в методе gameStart() и состоит из следующих фаз:

а) Начало игры — метод initGame() (строки 46-54). Создается игрок с начальными характеристиками (1000 HP, дистанция атаки 5) и рука из 5 случайных заклинаний.

б) Запуск уровня — метод loadLevel() (строки 56-68). Создается игровое поле размером, определенным текущим уровнем, размещается игрок, инициализируются враги, башни и здания согласно конфигурации уровня.

с) Ход игрока — метод playerTurn() (строки 335-382). Игрок выполняет действие (перемещение, атака или применение заклинания) через паттерн Command (класс GameAction). После хода игрока обновляются башни, проверяются ловушки и атаки башен. Реализован случайный шанс получения новой карты заклинания (20% после атаки).

d) Ход союзников — метод `alliesTurn()` (строки 325-329). Союзники автоматически перемещаются к ближайшим врагам через метод `moveAllies()` класса `EntityManager`.

e) Ход врагов — метод `enemiesTurn()` (строки 331-335). Враги автоматически перемещаются к игроку через метод `moveAllEnemies()` класса `EntityManager`.

f) Ход вражеской базы и башни — методы `towersTurn()` и `buildingsTurn()` (строки 337-345). Башни атакуют игрока при нахождении в радиусе с учетом кулдауна. Здания спавнят новых врагов через метод `processBuildingSpawns()`.

Все взаимодействие происходит через классы `GameControl`, `EntityManager` и систему действий `GameAction`. Команды пользователя обрабатываются через `InputHandler` и преобразуются в соответствующие действия.

2. Условия прохождения уровня и обработка поражения

Абстрактный класс `Level` создан (`core/Level.cpp`, `core/Level.h`). Класс определяет контракт для всех уровней через чисто виртуальные методы:

- `initialize(EntityManager&)` — инициализация уровня
- `isVictoryCondition(EntityManager&)` — условие победы
- `getName()`, `getDescription()` — информация об уровне
- `getBoardSize()` — размер поля

Реализовано 4 конкретных уровня:

- `Level1_Tutorial` — обучающий уровень (3 слабых врага, HP: 5, Damage: 1)
- `Level2_TowerDefense` — защита от башен
- `Level3_BuildingAssault` — штурм зданий
- `Level4_FinalBattle` — финальная битва

Условия победы определяются индивидуально для каждого уровня.

Например, для Level1_Tutorial победа наступает при уничтожении всех врагов (метод isVictoryCondition() возвращает em.getAliveEnemyCount() == 0).

Проверка победы реализована в методе isVictory() (строки 304-309), который вызывает метод isVictoryCondition() текущего уровня.

Обработка поражения реализована в методе isGameOver() (строки 395-397), который проверяет здоровье игрока. При поражении игроку предлагается:

- Начать заново с 1000 HP и первого уровня (при выборе 'y')
- Восстановить HP и перезапустить текущий уровень (при выборе 'n')

Реализация находится в основном цикле gameStart() (строки 120-165).

3. Система сохранения и загрузки игры

Класс GameSaveManager создан (core/GameSaveManager.cpp, core/GameSaveManager.h). Класс реализует статические методы для сохранения и загрузки состояния игры в бинарном формате.

Формат сохранения:

- Заголовок-сигнатура "GAME_SAVE_V1" для проверки корректности файла
- Счетчик ходов (countMove)
- Индекс текущего уровня
- Состояние игрока (HP, урон, дистанция, статусы, очки улучшений)
- Размер игрового поля
- EntityManager (враги, союзники, башни, здания, ловушки с координатами)
- Рука заклинаний (названия и количество каждого заклинания)

Методы сохранения:

- saveGame() — основной метод сохранения (строки 33-81)
- savePlayer() — сохранение данных игрока (строки 95-103)
- saveEntityManager() — сохранение всех сущностей (строки 130-187)
- saveHand() — сохранение заклинаний (строки 272-289)

Методы загрузки:

- loadGame() — основной метод загрузки (строки 83-155)
- loadPlayer() — восстановление состояния игрока (строки 105-128)
- loadEntityManager() — восстановление сущностей (строки 189-270)
- loadHand() — восстановление заклинаний через SpellFactory (строки 291-323)

Интеграция в игровой цикл:

- Сохранение доступно по команде "save" в любой момент игры
- Загрузка доступна по команде "load" в игре или через главное меню
- Проверка наличия сохранения через метод saveExists()

Консистентность данных обеспечивается через:

- RAII-подход (автоматическое закрытие файлов)
- Атомарность операций (режим std::ios::trunc)
- Проверка загруженного состояния перед продолжением игры

4. Обработка исключительных ситуаций для загрузки/сохранения

Создана иерархия исключений (core/GameExceptions.h):

- `GameException` — базовое исключение, наследуется от `std::runtime_error`, содержит поле `location` для указания места возникновения ошибки
- `SaveException` — исключение для ошибок сохранения
- `LoadException` — исключение для ошибок загрузки
- `InvalidDataException` — исключение для некорректных данных (наследуется от `LoadException`)
- `FileNotFoundException` — исключение для отсутствия файла (наследуется от `LoadException`)

Обработка исключений при сохранении (строки 33-81 в `GameSaveManager.cpp`):

- Проверка возможности открытия файла для записи
- При ошибке выбрасывается `SaveException` с описанием ошибки
- Гарантированное закрытие файла через блок `try-catch`

Обработка исключений при загрузке (строки 83-155 в `GameSaveManager.cpp`):

- Проверка существования файла (`FileNotFoundException`)
- Проверка корректности заголовка (`InvalidDataException`)
- Проверка корректности данных при загрузке `EntityManager` (защита от некорректных значений: слишком большое количество врагов, отрицательные координаты, отрицательное здоровье)
- Гарантированное закрытие файла через блок `try-catch`

Обработка исключений в главном меню (строки 487-497 в `GameControl.cpp`):

- Перехват `LoadException` при загрузке игры
- Вывод сообщения об ошибке пользователю
- Возврат в главное меню без завершения программы

6. Реализация перехода на следующий уровень

Класс LevelManager создан (core/LevelManager.cpp, core/LevelManager.h).

Класс управляет коллекцией уровней и переходами между ними.

Структура LevelManager:

- vector<unique_ptr<Level>> levels — коллекция всех уровней игры
- Level* currentLevel — указатель на текущий уровень
- int currentLevelIndex — индекс текущего уровня

Методы управления уровнями:

- createLevels() — создание всех уровней игры (строки 16-23). Создается 4 уровня: Level1_Tutorial, Level2_TowerDefense, Level3_BuildingAssault, Level4_FinalBattle

- loadLevel(int index) — загрузка уровня по индексу (строки 25-33)
- nextLevel() — переход на следующий уровень (строки 35-41)
- hasNextLevel() — проверка наличия следующего уровня
- isLastLevel() — проверка, является ли текущий уровень последним

Проверка победы и переход реализованы в основном цикле (строки 167-180 в GameController.cpp):

- При победе вызывается showLevelComplete()
- Проверяется наличие следующего уровня через hasNextLevel()
- Вызывается prepareForNextLevel() для подготовки игрока
- Вызывается nextLevel() для перехода
- При отсутствии следующего уровня вызывается showGameComplete()

Подготовка к следующему уровню (метод prepareForNextLevel()):

- Восстановление здоровья игрока через player->RestoreHealth()
- Удаление половины карт заклинаний случайным образом

Характеристики уровней:

- Каждый уровень определяет размер поля через `getBoardSize()`
- Каждый уровень определяет сложность через `getDifficulty()` (от 1 до 5)
- Каждый уровень определяет начальное расположение врагов через `initialize()`
- С каждым уровнем увеличивается количество и сила врагов

7. Реализация прокачки игрока при переходе между уровнями

Система прокачки реализована в классе `Player`. Состояние прокачки сохраняется и загружается через `GameSaveManager`.

Очки улучшений (`upgradePoints`):

- Сохраняются в методе `savePlayer()` (строки 95-103)
- Загружаются в методе `loadPlayer()` (строки 105-128)
- Хранятся в классе `Player` и доступны через `GetUpgradePoints()` и `SetUpgradePoints()`

Механика прокачки:

- Увеличение урона через `SetBaseDamage()`
- Увеличение дистанции атаки через `SetDistance()` или `ToggleDoubleDistance()`
- Увеличение максимального здоровья

Расширение игрового цикла:

- Меню прокачки вызывается при переходе между уровнями (после `showLevelComplete()` и перед `prepareForNextLevel()`)
- Игроку предлагается выбрать улучшение персонажа
- Каждое улучшение стоит определенное количество очков

Получение очков прокачки:

- Очки начисляются при прохождении уровня
- Количество очков зависит от сложности уровня (getDifficulty())

Система улучшения заклинаний:

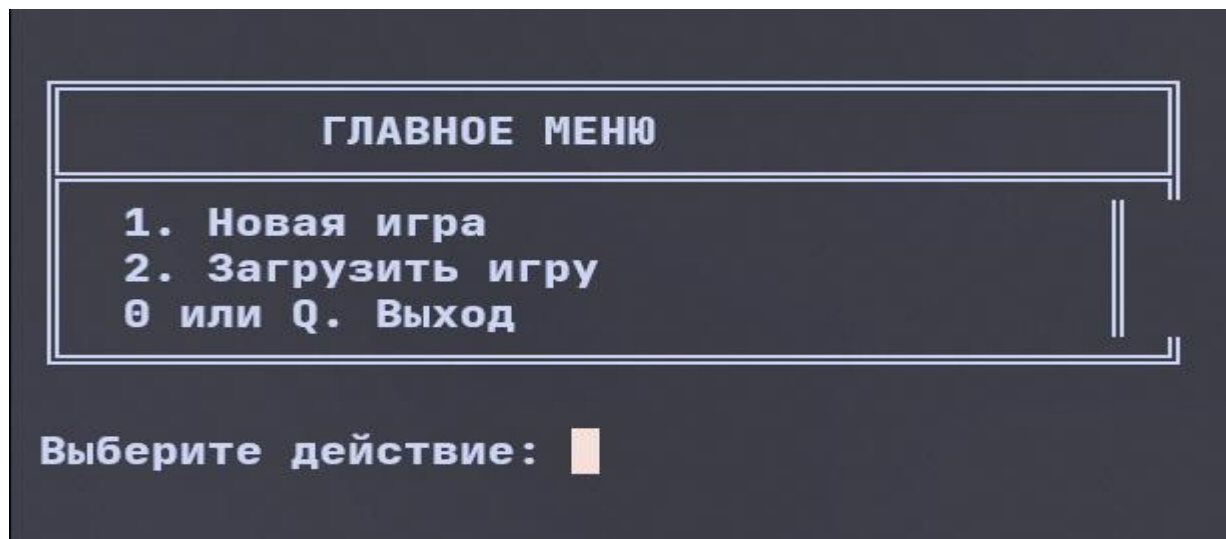
- Реализована через класс EnhanceSpell
- Позволяет улучшить следующее используемое заклинание
- Увеличивает радиус для DirectDamage
- Увеличивает площадь для AreaDamage
- Увеличивает урон для TrapSpell
- Увеличивает количество призванных союзников для SummonSpell
- Поддерживает накопление усилений (можно использовать несколько раз подряд)

ИСПОЛЬЗОВАННЫЕ ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ:

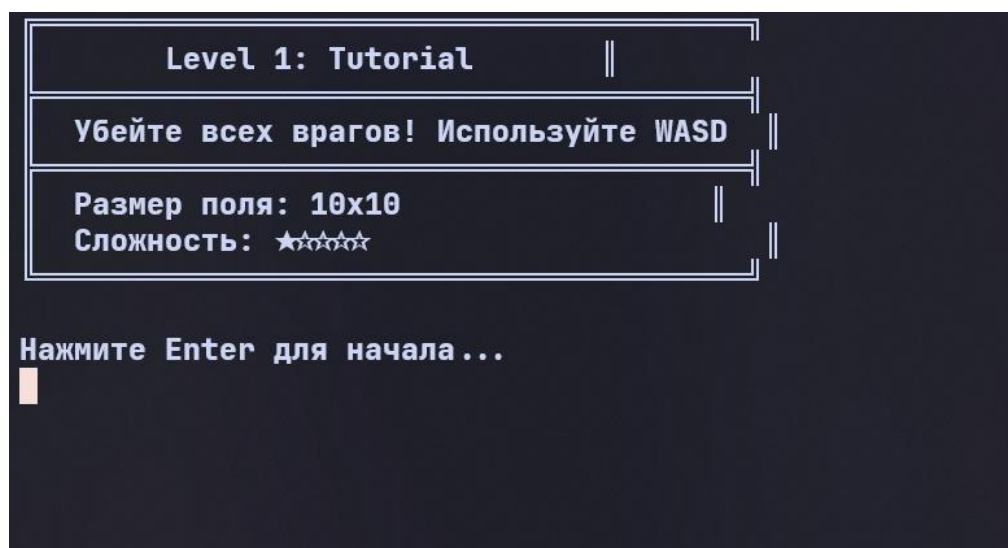
1. Command — класс GameAction для инкапсуляции действий игрока
2. Factory — SpellFactory для создания заклинаний при загрузке
3. Strategy — Level::isVictoryCondition() для различных условий победы
4. RAII — unique_ptr для автоматического управления памятью
5. Template Method — абстрактный Level с конкретными реализациями

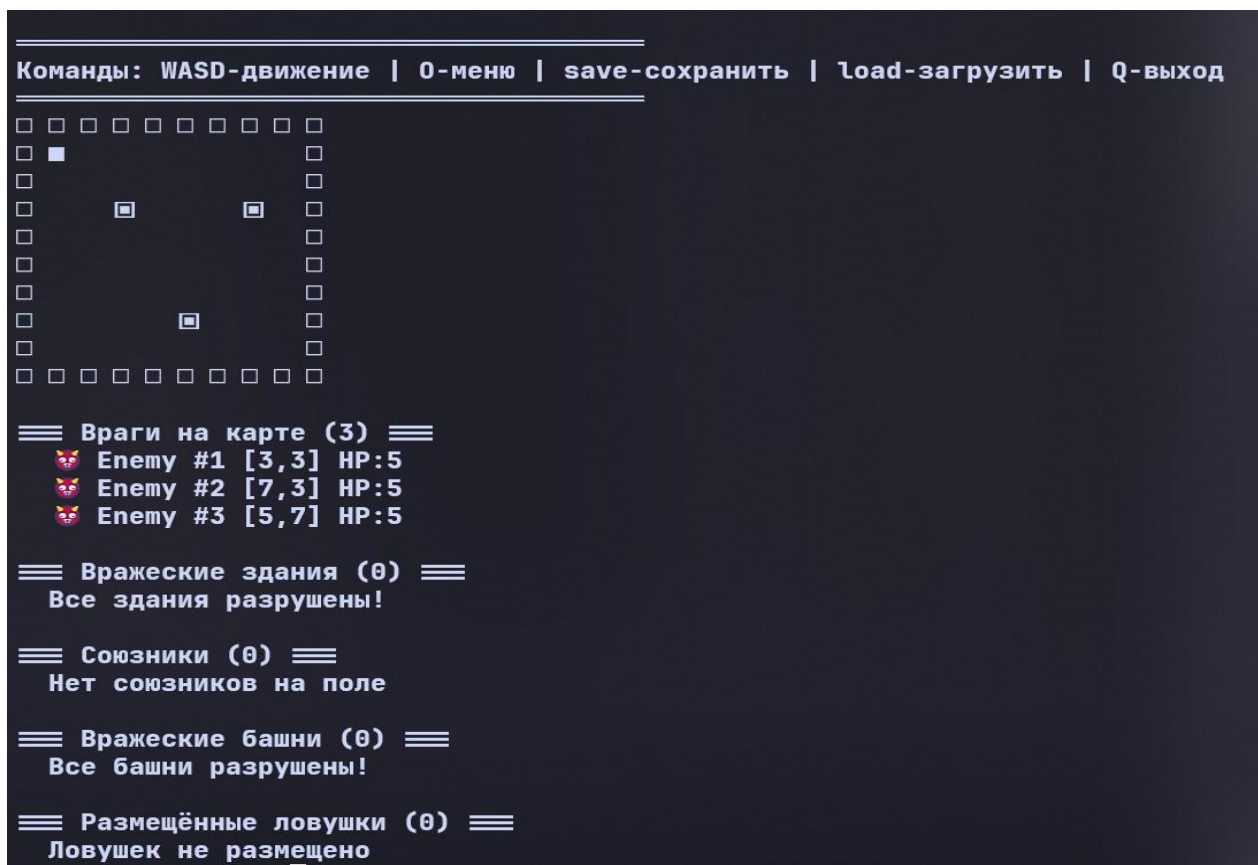
Тестирование:

0. Начало игры

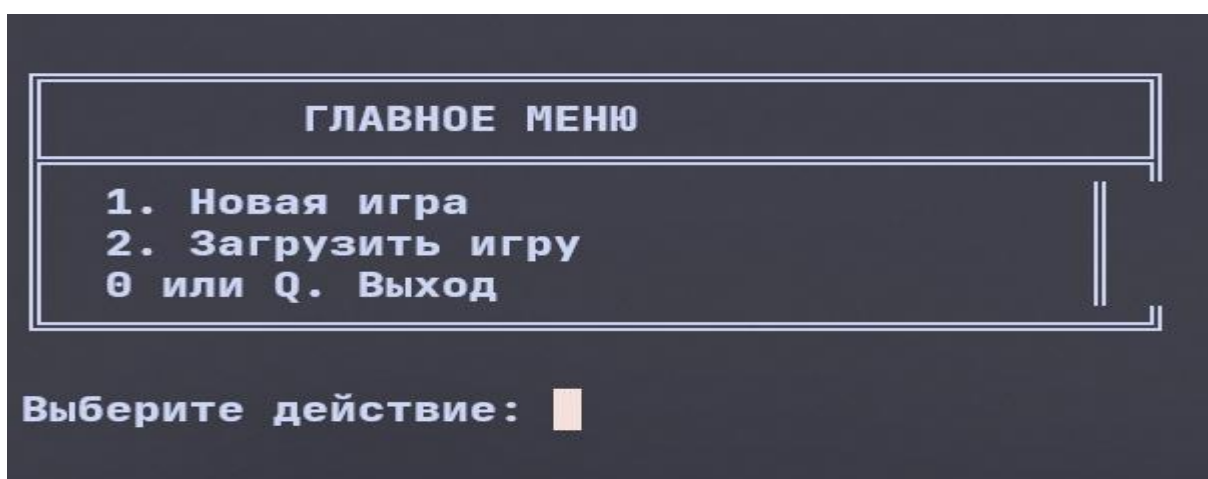


1. Запуск уровня





2. Главный экран и запуск 4 уровня который был сохранен ранее



```

Выберите действие: 2
Board created
Враг #1 создан в (5, 5)
Враг #2 создан в (20, 5)
Враг #3 создан в (5, 20)
Враг #4 создан в (20, 20)
Враг #5 создан в (12, 12)
🏰 Враждебная башня добавлена на (10, 10)
🏰 Враждебная башня добавлена на (15, 10)
🏰 Враждебная башня добавлена на (10, 15)
[DEBUG] Вражеское здание #1 создано в (18, 18) с интервалом спавна 3 ходов
[DEBUG] Grid установлен: getCell(18, 18)=4 (должно быть HellBuild=4)
[DEBUG] Вражеское здание #2 создано в (22, 22) с интервалом спавна 3 ходов
[DEBUG] Grid установлен: getCell(22, 22)=4 (должно быть HellBuild=4)

✅ Игра успешно загружена!
Нажмите Enter для продолжения...

```

```

≡ Player Stats ≡
Health: 1000 / 5
Damage: 17
Distance: 1
=====
🎯 Цели: Враги: 5 | Здания: 2

| × Нет доступных заклинаний |

Команды: WASD-движение | O-меню | save-сохранить | load-загрузить | Q-выход

[Map Grid]
[Enemy List]
≡ Враги на карте (5) ≡
👾 Enemy #1 [5,5] HP:15
👾 Enemy #2 [20,5] HP:15
👾 Enemy #3 [5,20] HP:15
👾 Enemy #4 [20,20] HP:15
👾 Enemy #5 [12,12] HP:15

```

3. Восстановление хп, переход на другой уровень и половина карточек случайно удаляется

УРОВЕНЬ ПРОЙДЕН! 🎉

Level 1: Tutorial ||

Ходов: 18 ||

HP: 999 / 1000 ||

Переход к следующему уровню...

Нажмите Enter для продолжения...

💎 HP восстановлено до максимума!

☀️ Получено очков прокачки: 3

💎 Всего очков прокачки: 3

🌀 Половина карт заклинаний была утеряна! (2 карт)

🔑 Используйте меню опций (0) для прокачки персонажа!
Нажмите Enter для продолжения...

≡ Player Stats ≡

Health: 1000 / 1000

Damage: 5

Distance: 1

=====

 Цели: Враги: 4 | Здания: 0

[0] ⚡ Summon Spell
[1] ⚡ Enhance Spell
[2] (пустой слот)
[3] (пустой слот)
[4] (пустой слот)

4. Прокачка игрока

☀ ПРОКАЧКА ПЕРСОНАЖА ☀

Доступно очков: 3

1 - Увеличить макс. HP (+100) [1]
2 - Увеличить урон (+2) [1]
3 - Улучшить случайное заклинание
(+50% урон/радиус) [2]
ESC - Вернуться

Текущие характеристики:

HP: 1000 / 1000

Урон: 5

Заклинаний в руке: 3

Введите опцию:

Все очки направлены на увеличение урона

☀ ПРОКАЧКА ПЕРСОНАЖА ☀

Доступно очков: 0

- 1 - Увеличить макс. HP (+100) [1]
- 2 - Увеличить урон (+2) [1]
- 3 - Улучшить случайное заклинание
(+50% урон/радиус) [2]
- ESC - Вернуться

Текущие характеристики:

HP: 1000 / 1000

Урон: 11

Заклинаний в руке: 3

Введите опцию:

Вывод.

Все требования лабораторной работы выполнены. Реализован полноценный игровой цикл с поддержкой пошаговой механики, система уровней с различными условиями победы, надежная система сохранения и загрузки с обработкой исключений, переходы между уровнями с изменением сложности и система прокачки игрока. Код соответствует современным стандартам C++ и использует хорошие практики объектно-ориентированного программирования